

32 数据类型决定了程序中数据和操作的意义。如下所示的语句是一个简单示例：

```
i = i + j;
```

其含义依赖于 `i` 和 `j` 的数据类型。如果 `i` 和 `j` 都是整数数，那么这条语句执行的就是最普通的加法运算。然而，如果 `i` 和 `j` 是 `Sales_item` 类型的数据（参见 1.5.1 节，第 17 页），则上述语句把这两个对象的成分相加。

2.1 基本内置类型

C++定义了一套包括算术类型（arithmetic type）和空类型（void）在内的基本数据类型。其中算术类型包含了字符、整型数、布尔值和浮点数。空类型不对应具体的值，仅用于一些特殊的场合，例如最常见的是，当函数不返回任何值时使用空类型作为返回类型。

2.1.1 算术类型

算术类型分为两类：整型（integral type，包括字符和布尔类型在内）和浮点型。

算术类型的尺寸（也就是该类型数据所占的比特数）在不同机器上有所差别。表 2.1 列出了 C++标准规定的尺寸的最小值，同时允许编译器赋予这些类型更大的尺寸。某一类型所占的比特数不同，它所能表示的数据范围也不一样。

表 2.1: C++: 算术类型		
类型	含义	最小尺寸
bool	布尔类型	未定义
char	字符	8 位
wchar_t	宽字符	16 位
char16_t	Unicode 字符	16 位
char32_t	Unicode 字符	32 位
short	短整型	16 位
int	整型	16 位
long	长整型	32 位
long long	长整型	64 位
float	单精度浮点数	6 位有效数字
double	双精度浮点数	10 位有效数字
long double	扩展精度浮点数	10 位有效数字

布尔类型（bool）的取值是真（true）或者假（false）。

C++提供了几种字符类型，其中多数支持国际化。基本的字符类型是 `char`，一个 `char` 的空间应确保可以存放机器基本字符集中任意字符对应的数字值。也就是说，一个 `char` 的大小和一个机器字节一样。

33 其他字符类型用于扩展字符集，如 `wchar_t`、`char16_t`、`char32_t`。`wchar_t` 类型用于确保可以存放机器最大扩展字符集中的任意一个字符，类型 `char16_t` 和 `char32_t` 则为 Unicode 字符集服务（Unicode 是用于表示所有自然语言中字符的标准）。

除字符和布尔类型之外，其他整型用于表示（可能）不同尺寸的整数。C++语言规定一个 `int` 至少和一个 `short` 一样大，一个 `long` 至少和一个 `int` 一样大，一个 `long long`

至少和一个 long 一样大。其中，数据类型 long long 是在 C++11 中新定义的。

内置类型的机器实现

计算机以比特序列存储数据，每个比特非 0 即 1，例如：

```
00011011011100010110010000111011 ...
```

大多数计算机以 2 的整数次幂个比特作为块来处理内存，可寻址的最小内存块称为“字节 (byte)”，存储的基本单元称为“字 (word)”，它通常由几个字节组成。在 C++ 语言中，一个字节要至少能容纳机器基本字符集中的字符。大多数机器的字节由 8 比特构成，字则由 32 或 64 比特构成，也就是 4 或 8 字节。

大多数计算机将内存中的每个字节与一个数字（被称为“地址 (address)”）关联起来，在一个字节为 8 比特、字为 32 比特的机器上，我们可能看到一个字的内存区域如下所示：

736424	0	0	1	1	1	0	1	1
736425	0	0	0	1	1	0	1	1
736426	0	1	1	1	0	0	0	1
736427	0	1	1	0	0	1	0	0

其中，左侧是字节的地址，右侧是字节中 8 比特的具体内容。

我们能够使用某个地址来表示从这个地址开始的大小不同的比特串，例如，我们可能会说地址 736424 的那个字或者地址 736427 的那个字节。为了赋予内存中某个地址明确的含义，必须首先知道存储在该地址的数据的类型。类型决定了数据所占的比特数以及该如何解释这些比特的内容。

如果位置 736424 处的对象类型是 float，并且该机器中 float 以 32 比特存储，那么我们就能知道这个对象的内容占满了整个字。这个 float 数的实际值依赖于该机器是如何存储浮点数的。或者如果位置 736424 处的对象类型是 unsigned char，并且该机器使用 ISO-Latin-1 字符集，则该位置处的字节表示一个分号。

浮点型可表示单精度、双精度和扩展精度值。C++ 标准指定了一个浮点数有效位数的最小值，然而大多数编译器都实现了更高的精度。通常，float 以 1 个字（32 比特）来表示，double 以 2 个字（64 比特）来表示，long double 以 3 或 4 个字（96 或 128 比特）来表示。一般来说，类型 float 和 double 分别有 7 和 16 个有效位；类型 long double 则常常被用于有特殊浮点需求的硬件，它的具体实现不同，精度也各不相同。

带符号类型和无符号类型

除去布尔型和扩展的字符型之外，其他整型可以划分为带符号的 (signed) 和无符号的 (unsigned) 两种。带符号类型可以表示正数、负数或 0，无符号类型则仅能表示大于等于 0 的值。

类型 int、short、long 和 long long 都是带符号的，通过在这些类型名前添加 unsigned 就可以得到无符号类型，例如 unsigned long。类型 unsigned int 可以缩写为 unsigned。

与其他整型不同，字符型被分为了三种：char、signed char 和 unsigned char。

特别需要注意的是：类型 char 和类型 signed char 并不一样。尽管字符型有三种，但是字符的表现形式却只有两种：带符号的和无符号的。类型 char 实际上会表现为上述两种形式中的一种，具体是哪种由编译器决定。

无符号类型中所有比特都用来存储值，例如，8 比特的 unsigned char 可以表示 0 至 255 区间内的值。

C++ 标准并没有规定带符号类型应如何表示，但是约定了在表示范围内正值和负值的量应该平衡。因此，8 比特的 signed char 理论上应该可以表示 -127 至 127 区间内的值，大多数现代计算机将实际的表示范围定为 -128 至 127。

建议：如何选择类型

和 C 语言一样，C++ 的设计准则之一也是尽可能地接近硬件。C++ 的算术类型必须满足各种硬件特质，所以它们常常显得繁杂而令人不知所措。事实上，大多数程序员能够（也应该）对数据类型的使用做出限定从而简化选择的过程。以下是选择类型的一些经验准则：

- 当明确知晓数值不可能为负时，选用无符号类型。
- 使用 int 执行整数运算。在实际应用中，short 常常显得太小而 long 一般和 int 有一样的尺寸。如果你的数值超过了 int 的表示范围，选用 long long。
- 在算术表达式中不要使用 char 或 bool，只有在存放字符或布尔值时才使用它们。因为类型 char 在一些机器上是有符号的，而在另一些机器上又是无符号的，所以如果使用 char 进行运算特别容易出问题。如果你需要使用一个不大的整数，那么明确指定它的类型是 signed char 或者 unsigned char。
- 执行浮点数运算选用 double，这是因为 float 通常精度不够而且双精度浮点数和单精度浮点数的计算代价相差无几。事实上，对于某些机器来说，双精度运算甚至比单精度还快。long double 提供的精度在一般情况下是没有必要的，况且它带来的运行时消耗也不容忽视。

35

2.1.1 节练习

练习 2.1：类型 int、long、long long 和 short 的区别是什么？无符号类型和带符号类型的区别是什么？float 和 double 的区别是什么？

练习 2.2：计算按揭贷款时，对于利率、本金和付款分别应选择何种数据类型？说明你的理由。



2.1.2 类型转换

对象的类型定义了对对象能包含的数据和能参与的运算，其中一种运算被大多数类型支持，就是将对象从一种给定的类型转换（convert）为另一种相关类型。

当在程序的某处我们使用了一种类型而其实对象应该取另一种类型时，程序会自动进行类型转换，在 4.11 节（第 141 页）中我们将对类型转换做更详细的介绍。此处，有必要说明当给某种类型的对象强行赋了另一种类型的值时，到底会发生什么。

当我们像下面这样把一种算术类型的值赋给另外一种类型时：

```
bool b = 42;           // b 为真
```



```

int i = b;           // i 的值为 1
i = 3.14;           // i 的值为 3
double pi = i;       // pi 的值为 3.0
unsigned char c = -1; // 假设 char 占 8 比特, c 的值为 255
signed char c2 = 256; // 假设 char 占 8 比特, c2 的值是未定义的

```

类型所能表示的值的范围决定了转换的过程:

- 当我们把一个非布尔类型的算术值赋给布尔类型时, 初始值为 0 则结果为 false, 否则结果为 true。
- 当我们把一个布尔值赋给非布尔类型时, 初始值为 false 则结果为 0, 初始值为 true 则结果为 1。
- 当我们把一个浮点数赋给整数类型时, 进行了近似处理。结果值将仅保留浮点数中小数点之前的部分。
- 当我们把一个整数值赋给浮点类型时, 小数部分记为 0。如果该整数所占的空间超过了浮点类型的容量, 精度可能有损失。
- 当我们赋给无符号类型一个超出它表示范围的值时, 结果是初始值对无符号类型表示数值总数取模后的余数。例如, 8 比特大小的 unsigned char 可以表示 0 至 255 区间内的值, 如果我们赋了一个区间以外的值, 则实际的结果是该值对 256 取模后所得的余数。因此, 把 -1 赋给 8 比特大小的 unsigned char 所得的结果是 255。
- 当我们赋给带符号类型一个超出它表示范围的值时, 结果是未定义的 (undefined)。此时, 程序可能继续工作、可能崩溃, 也可能生成垃圾数据。

建议: 避免无法预知和依赖于实现环境的行为

36

无法预知的行为源于编译器无须 (有时是不能) 检测的错误。即使代码编译通过了, 如果程序执行了一条未定义的表达式, 仍有可能产生错误。

不幸的是, 在某些情况和/或某些编译器下, 含有无法预知行为的程序也能正确执行。但是我们却无法保证同样一个程序在别的编译器下能正常工作, 甚至已经编译通过的代码再次执行也可能会出错。此外, 也不能认为这样的程序对一组输入有效, 对另一组输入就一定有效。

程序也应该尽量避免依赖于实现环境的行为。如果我们把 int 的尺寸看成是一个确定不变的已知值, 那么这样的程序就称作不可移植的 (nonportable)。当程序移植到别的机器上后, 依赖于实现环境的程序就可能发生错误。要从过去的代码中定位这类错误可不是一件轻松愉快的工作。

当在程序的某处使用了一种算术类型的值而其实所需的是另一种类型的值时, 编译器同样会执行上述的类型转换。例如, 如果我们使用了一个非布尔值作为条件 (参见 1.4.1 节, 第 10 页), 那么它会被自动地转换成布尔值, 这一做法和把非布尔值赋给布尔变量时的操作完全一样:

```

int i = 42;
if (i)           // if 条件的值将为 true
    i = 0;

```

如果 i 的值为 0, 则条件的值为 false; i 的所有其他取值 (非 0) 都将使条件为 true。

以此类推，如果我们把一个布尔值用在算术表达式里，则它的取值非 0 即 1，所以一般不宜在算术表达式里使用布尔值。



含有无符号类型的表达式

尽管我们不会故意给无符号对象赋一个负值，却可能（特别容易）写出这么做的代码。例如，当一个算术表达式中既有无符号数又有 `int` 值时，那个 `int` 值就会转换成无符号数。把 `int` 转换成无符号数的过程 and 把 `int` 直接赋给无符号变量一样：

```
unsigned u = 10;
int i = -42;
std::cout << i + i << std::endl; // 输出-84
std::cout << u + i << std::endl; // 如果 int 占 32 位，输出 4294967264
```

在第一个输出表达式里，两个（负）整数相加并得到了期望的结果。在第二个输出表达式里，相加前首先把整数 -42 转换成无符号数。把负数转换成无符号数类似于直接给无符号数赋一个负值，结果等于这个负数加上无符号数的模。

当从无符号数中减去一个值时，不管这个值是不是无符号数，我们都必须确保结果不能是一个负值：

37

```
unsigned u1 = 42, u2 = 10;
std::cout << u1 - u2 << std::endl; // 正确：输出 32
std::cout << u2 - u1 << std::endl; // 正确：不过，结果是取模后的值
```

无符号数不会小于 0 这一事实同样关系到循环的写法。例如，在 1.4.1 节的练习（第 11 页）中需要写一个循环，通过控制变量递减的方式把从 10 到 0 的数字降序输出。这个循环可能类似于下面的形式：

```
for (int i = 10; i >= 0; --i)
    std::cout << i << std::endl;
```

可能你会觉得反正也不打算输出负数，可以用无符号数来重写这个循环。然而，这个不经意的改变却意味着死循环：

```
// 错误：变量 u 永远也不会小于 0，循环条件一直成立
for (unsigned u = 10; u >= 0; --u)
    std::cout << u << std::endl;
```

来看看当 `u` 等于 0 时发生了什么，这次迭代输出 0，然后继续执行 `for` 语句里的表达式。表达式 `--u` 从 `u` 当中减去 1，得到的结果 -1 并不满足无符号数的要求，此时像所有表示范围之外的其他数字一样，-1 被自动地转换成一个合法的无符号数。假设 `int` 类型占 32 位，则当 `u` 等于 0 时，`--u` 的结果将会是 4294967295。

一种解决的办法是，用 `while` 语句来代替 `for` 语句，因为前者让我们能够在输出变量之前（而非之后）先减去 1：

```
unsigned u = 11; // 确定要输出的最大数，从比它大 1 的数开始
while (u > 0){
    --u;          // 先减 1，这样最后一次迭代时就会输出 0
    std::cout << u << std::endl;
}
```

改写后的循环先执行对循环控制变量减 1 的操作，这样最后一次迭代时，进入循环的 `u` 值为 1。此时将其减 1，则这次迭代输出的数就是 0；下一次再检验循环条件时，`u` 的值等

于 0 而无法再进入循环。因为我们要先做减 1 的操作，所以初始化 `u` 的值应该比要输出的最大值大 1。这里，`u` 初始化为 11，输出的最大数是 10。

提示：切勿混用带符号类型和无符号类型

如果表达式里既有带符号类型又有无符号类型，当带符号类型取值为负时会出现异常结果，这是因为带符号数会自动地转换成无符号数。例如，在一个形如 `a*b` 的式子中，如果 `a = -1`，`b = 1`，而且 `a` 和 `b` 都是 `int`，则表达式的值显然为 `-1`。然而，如果 `a` 是 `int`，而 `b` 是 `unsigned`，则结果须视在当前机器上 `int` 所占位数而定。在我们的环境里，结果是 4294967295。

2.1.2 节练习

38

练习 2.3：读程序写结果。

```
unsigned u = 10, u2 = 42;
std::cout << u2 - u << std::endl;
std::cout << u - u2 << std::endl;

int i = 10, i2 = 42;
std::cout << i2 - i << std::endl;
std::cout << i - i2 << std::endl;
std::cout << i - u << std::endl;
std::cout << u - i << std::endl;
```

练习 2.4：编写程序检查你的估计是否正确，如果不正确，请仔细研读本节直到弄明白问题所在。

2.1.3 字面值常量

一个形如 42 的值被称作字面值常量 (literal)，这样的值一望而知。每个字面值常量都对应一种数据类型，字面值常量的形式和值决定了它的数据类型。

整型和浮点型字面值

我们可以将整型字面值写作十进制数、八进制数或十六进制数的形式。以 0 开头的整数代表八进制数，以 0x 或 0X 开头的代表十六进制数。例如，我们能用下面的任意一种形式来表示数值 20：

```
20 /* 十进制 */      024 /* 八进制 */      0x14 /* 十六进制 */
```

整型字面值具体的数据类型由它的值和符号决定。默认情况下，十进制字面值是带符号数，八进制和十六进制字面值既可能是带符号的也可能是无符号的。十进制字面值的类型是 `int`、`long` 和 `long long` 中尺寸最小的那个（例如，三者当中最小是 `int`），当然前提是这种类型要能容纳下当前的值。八进制和十六进制字面值的类型是能容纳其数值的 `int`、`unsigned int`、`long`、`unsigned long`、`long long` 和 `unsigned long long` 中的尺寸最小者。如果一个字面值连与之关联的最大的数据类型都放不下，将产生错误。类型 `short` 没有对应的字面值。在表 2.2（第 37 页）中，我们将以后缀代表相应的字面值类型。

尽管整型字面值可以存储在带符号数据类型中，但严格来说，十进制字面值不会是负

数。如果我们使用了一个形如-42 的负十进制字面值，那个负号并不在字面值之内，它的作用仅仅是对字面值取负值而已。

浮点型字面值表现为一个小数或以科学计数法表示的指数，其中指数部分用 E 或 e 标识：

```
3.14159      3.14159E0      0.      0e0      .001
```

39 默认的，浮点型字面值是一个 double，我们可以使用表 2.2（第 37 页）中的后缀来表示其他浮点型。

字符和字符串字面值

由单引号括起来的一个字符称为 char 型字面值，双引号括起来的零个或多个字符则构成字符串型字面值。

```
'a'          // 字符字面值
"Hello World!" // 字符串字面值
```

字符串字面值的类型实际上是由常量字符构成的数组（array），该类型将在 3.5.4 节（第 109 页）介绍。编译器在每个字符串的结尾处添加一个空字符（'\0'），因此，字符串字面值的实际长度要比它的内容多 1。例如，字面值'A'表示的就是单独的字符 A，而字符串"A"则代表了一个字符的数组，该数组包含两个字符：一个是字母 A、另一个是空字符。

如果两个字符串字面值位置紧邻且仅由空格、缩进和换行符分隔，则它们实际上是一个整体。当书写的字符串字面值比较长，写在一行里不太合适时，就可以采取分开书写的方式：

```
// 分多行书写的字符串字面值
std::cout << "a really, really long string literal "
              "that spans two lines" << std::endl;
```

转义序列

有两类字符程序员不能直接使用：一类是不可打印（nonprintable）的字符，如退格或其他控制字符，因为它们没有可视的图符；另一类是在 C++ 语言中有特殊含义的字符（单引号、双引号、问号、反斜线）。在这些情况下需要用到转义序列（escape sequence），转义序列均以反斜线作为开始，C++ 语言规定的转义序列包括：

换行符	\n	横向制表符	\t	报警（响铃）符	\a
纵向制表符	\v	退格符	\b	双引号	\"
反斜线	\\	问号	\?	单引号	\'
回车符	\r	进纸符	\f		

在程序中，上述转义序列被当作一个字符使用：

```
std::cout << '\n';          // 转到新一行
std::cout << "\tHi!\n";     // 输出一个制表符，输出"Hi!"，转到新一行
```

我们也可以使用泛化的转义序列，其形式是 \x 后紧跟 1 个或多个十六进制数字，或者 \后紧跟 1 个、2 个或 3 个八进制数字，其中数字部分表示的是字符对应的数值。假设使用的是 Latin-1 字符集，以下是一些示例：

```
\7（响铃）    \12（换行符）    \40（空格）
\0（空字符）  \115（字符M）    \x4d（字符M）
```

40 我们可以像使用普通字符那样使用 C++ 语言定义的转义序列：

```
std::cout << "Hi \x4d0\115!\n"; // 输出 Hi MOM!，转到新一行
```

```
std::cout << '\115' << '\n';    //输出 M，转到新一行
```

注意，如果反斜线\后面跟着的八进制数字超过 3 个，只有前 3 个数字与\构成转义序列。例如，"\1234"表示 2 个字符，即八进制数 123 对应的字符以及字符 4。相反，\x 要用到后面跟着的所有数字，例如，"\x1234"表示一个 16 位的字符，该字符由这 4 个十六进制数所对应的比特唯一确定。因为大多数机器的 char 型数据占 8 位，所以上面这个例子可能会报错。一般来说，超过 8 位的十六进制字符都是与表 2.2 中某个前缀作为开头的扩展字符集一起使用的。

指定字面值的类型

通过添加如表 2.2 中所列的前缀和后缀，可以改变整型、浮点型和字符型字面值的默认类型。

```
L'a'           // 宽字符型字面值，类型是 wchar_t
u8"hi!"        // utf-8 字符串字面值 (utf-8 用 8 位编码一个 Unicode 字符)
42ULL          // 无符号整型字面值，类型是 unsigned long long
1E-3F          // 单精度浮点型字面值，类型是 float
3.14159L       // 扩展精度浮点型字面值，类型是 long double
```

Best Practices

当使用一个长整型字面值时，请使用大写字母 L 来标记，因为小写字母 l 和数字 1 太容易混淆了。

表 2.2: 指定字面值的类型			
字符和字符串字面值			
前缀	含义	类型	
u	Unicode 16 字符	char16_t	
U	Unicode 32 字符	char32_t	
L	宽字符	wchar_t	
u8	UTF-8 (仅用于字符串字面常量)	char	
整型字面值		浮点型字面值	
后缀	最小匹配类型	后缀	类型
u or U	unsigned	f 或 F	float
l or L	long	l 或 L	long double
ll or LL	long long		

对于一个整型字面值来说，我们能分别指定它是否带符号以及占用多少空间。如果后缀中有 u，则该字面值属于无符号类型，也就是说，以 u 为后缀的十进制数、八进制数或十六进制数都将从 unsigned int、unsigned long 和 unsigned long long 中选择能匹配的空间最小的一个作为其数据类型。如果后缀中有 L，则字面值的类型至少是 long；如果后缀中有 LL，则字面值的类型将是 long long 和 unsigned long long 中的一种。显然我们可以将 u 与 L 或 LL 合在一起使用。例如，以 UL 为后缀的字面值的数据类型将根据具体数值情况或者取 unsigned long，或者取 unsigned long long。

布尔字面值和指针字面值

```
true 和 false 是布尔类型的字面值:

bool test = false;
```


`nullptr` 是指针字面值，2.3.2 节（第 47 页）将有更多关于指针和指针字面值的介绍。

2.1.3 节练习

练习 2.5：指出下述字面值的数据类型并说明每一组内几种字面值的区别：

- (a) `'a'`, `L'a'`, `"a"`, `L"a"`
- (b) `10`, `10u`, `10L`, `10uL`, `012`, `0xC`
- (c) `3.14`, `3.14f`, `3.14L`
- (d) `10`, `10u`, `10.`, `10e-2`

练习 2.6：下面两组定义是否有区别，如果有，请叙述之：

```
int month = 9, day = 7;
int month = 09, day = 07;
```

练习 2.7： 字面值表示何种含义？它们各自的数据类型是什么？

- (a) `"Who goes with F\145rgus?\012"`
- (b) `3.14e1L` (c) `1024f` (d) `3.14L`

练习 2.8：请利用转义序列编写一段程序，要求先输出 2M，然后转到新一行。修改程序使其先输出 2，然后输出制表符，再输出 M，最后转到新一行。

2.2 变量

变量提供一个具名的、可供程序操作的存储空间。C++ 中的每个变量都有其数据类型，数据类型决定着变量所占内存空间的大小和布局方式、该空间能存储的值的范围，以及变量能参与的运算。对 C++ 程序员来说，“变量 (variable)” 和 “对象 (object)” 一般可以互换使用。



2.2.1 变量定义

变量定义的基本形式是：首先是类型说明符 (type specifier)，随后紧跟由一个或多个变量名组成的列表，其中变量名以逗号分隔，最后以分号结束。列表中每个变量名的类型都由类型说明符指定，定义时还可以为一个或多个变量赋初值：

```
int sum = 0, value, // sum、value 和 units_sold 都是 int
    units_sold = 0; // sum 和 units_sold 初值为 0
Sales_item item;    // item 的类型是 Sales_item (参见 1.5.1 节，第 17 页)
// string 是一种库类型，表示一个可变长的字符序列
std::string book("0-201-78345-X"); // book 通过一个 string 字面值初始化
```

`book` 的定义用到了库类型 `std::string`，像 `iostream` (参见 1.2 节，第 6 页) 一样，`string` 也是在命名空间 `std` 中定义的，我们将在第 3 章中对 `string` 类型做更详细的介绍。眼下，只需了解 `string` 是一种表示可变长字符序列的数据类型就可以了。C++ 库提供了几种初始化 `string` 对象的方法，其中一种是把字面值拷贝给 `string` 对象 (参见 2.1.3 节，第 36 页)，因此在上例中，`book` 被初始化为 `0-201-78345-X`。